

Sistemi Operativi – a.a. 2025/2026

prova di laboratorio
– 9 settembre 2026 –

Creare un programma `phrase-decoder.c` in linguaggio C che accetti invocazioni sulla riga di comando del tipo:

```
phrase-decoder <file-1> ... <file-N>
```

Il programma prende in input $N \geq 1$ file di testo, ciascuno contenente una lista di frasi "codificate": ogni riga rappresenta una frase in chiaro che è stata codificata sostituendo, lettera per lettera, ogni carattere secondo una tra tre possibili trasformazioni, e alterando inoltre la capitalizzazione delle parole. Il programma dovrà far transitare ciascuna frase attraverso quattro thread trasformatori specializzati, ciascuno responsabile di annullare una specifica trasformazione, fino a ricostruire la frase originale.

I quattro trasformatori, nell'ordine fisso in cui una frase deve attraversarli, sono i seguenti:

- **LOWER-BACK**: ogni lettera minuscola presente nella frase viene sostituita con la lettera che la precede nell'alfabeto inglese ($a \rightarrow z$, $b \rightarrow a$, $c \rightarrow b$, e così via, ciclicamente); le lettere maiuscole non vengono toccate;
- **LEET**: alcune cifre e simboli vengono sostituiti da lettere minuscole secondo la codifica [leetspeak](#), con la tabella fissa $0 \rightarrow o$, $1 \rightarrow i$, $3 \rightarrow e$, $7 \rightarrow t$, $@ \rightarrow a$, $\$ \rightarrow s$; qualunque altro carattere non compare mai nelle frasi di questo compito e non va gestito;
- **UPPER-FWD**: ogni lettera maiuscola presente nella frase viene sostituita con la lettera maiuscola che la segue nell'alfabeto inglese ($A \rightarrow B$, ..., $Z \rightarrow A$ ciclicamente); le lettere minuscole non vengono toccate;
- **TITLE-CASE**: in ciascuna parola della frase (le parole sono separate da spazi singoli), la lettera iniziale viene resa maiuscola e tutte le altre lettere della stessa parola vengono rese minuscole.

Ad esempio la frase codificata `RHtS3LH NODs@uHwj`, attraversando in ordine LOWER-BACK, LEET, UPPER-FWD e TITLE-CASE, si trasforma rispettivamente in `RHsS3LH NODr@tHvi`, poi in `RHsSeLH NODrathvi`, poi in `SIsteMI OPeRatiVi` e infine nella frase in chiaro `Sistemi Operativi`.

L'ordine di applicazione dei quattro trasformatori deve rispettare scrupolosamente quello indicato sopra.

Sono disponibili tre [file di esempio](#), ciascuno con una lista di frasi codificate: `encoded-list-1.txt`, `encoded-list-2.txt` ed `encoded-list-3.txt`.

Il programma dovrà creare N thread **lettori** (uno per ciascun file indicato sulla riga di comando) e 4 thread **trasformatori**, uno per ciascuna delle trasformazioni sopra descritte:

- N thread **LETTORI** (uno per file) che leggeranno il proprio file riga per riga; per ciascuna frase letta, il lettore dovrà farla decodificare interrogando in sequenza i quattro trasformatori (prima LOWER-BACK, poi LEET, poi UPPER-FWD, infine TITLE-CASE), passando ogni volta come richiesta il risultato ottenuto dal trasformatore precedente; una volta ottenuto il risultato finale, il lettore lo visualizzerà sullo standard output insieme alla frase originale;
- 4 thread **TRASFORMATORI** (LOWER-BACK, LEET, UPPER-FWD, TITLE-CASE), ciascuno dei quali riceverà le richieste di tutti i lettori e applicherà, una alla volta, la trasformazione di propria competenza.

Le strutture dati condivise saranno:

- uno **slot** per ciascun trasformatore (quindi 4 in totale), usato in modalità **rendezvous** da tutti i lettori: contiene la frase da trasformare, l'identificativo del lettore richiedente, il numero progressivo della frase all'interno del file del lettore, e un campo di stato con i valori `EMPTY`, `TO_PROCESS`, `DONE`. Un lettore che vuole servirsi di un trasformatore attende che il suo slot sia libero (`EMPTY`); quando lo è, vi deposita la frase corrente marcandolo come da elaborare (`TO_PROCESS`) e si mette in attesa che la trasformazione sia completata (`DONE`); il

trasformatore, dal canto suo, resta in attesa che il proprio slot venga marcato come da elaborare, applica sul posto la trasformazione di sua competenza alla frase depositata e marca lo slot come completato; il lettore, non appena si accorge del completamento, legge il risultato e riporta lo slot allo stato libero, in modo che un altro lettore in attesa possa a sua volta servirsene;

- mutex e **variabili condizione**: il loro numero, comunque minimale, e le modalità d'uso sono da determinare da parte dello studente;
- eventuali flag/contatori di fine-lavoro.

Il thread principale/padre si occuperà unicamente di creare tutti i thread, attenderne la terminazione e visualizzare un riepilogo finale con il numero di frasi decodificate per ciascun file.

L'uso di mutex e variabili condizione è obbligatorio: soluzioni che non li utilizzino saranno considerate non corrette. Tutti i thread dovranno terminare spontaneamente alla fine dei lavori e non si dovranno usare strutture dati con visibilità globale. È necessario rispettare fedelmente la struttura dell'output riportato nell'esempio a seguire. Codici sorgente con errori che bloccano la compilazione e pregiudicano la generazione del codice binario con compilatori standard (gcc o clang) non saranno valutati.

Tempo: 2 ore e 15 minuti

La struttura dell'output atteso è la seguente:

```
$ ./phrase-decoder encoded-list-1.txt encoded-list-2.txt encoded-list-3.txt
```

```
[MAIN] creazione di 3 thread lettori e 4 thread trasformatori
[READER-2] apertura del file 'encoded-list-2.txt'
[READER-1] apertura del file 'encoded-list-1.txt'
[READER-2] frase n.1: 'MfK Amv Cjq1MS0 Cj cmT'
[READER-1] frase n.1: 'V3 xHKm QpBl z0v'
[LOWER-BACK] frase n.1 di READER-2: 'MfK Amv Cjq1MS0 Cj cmT' -> 'MeK Alu Cip1MS0 Ci bLT'
[LOWER-BACK] frase n.1 di READER-1: 'V3 xHKm QpBl z0v' -> 'V3 wHKL QoBk y0u'
[READER-3] apertura del file 'encoded-list-3.txt'
[READER-3] frase n.1: 'dGj U@ Oj@MN wb m0o7@M0'
[LOWER-BACK] frase n.1 di READER-3: 'dGj U@ Oj@MN wb m0o7@M0' -> 'cGi U@ Oi@MN va l0n7@M0'
[LEET] frase n.1 di READER-2: 'MeK Alu Cip1MS0 Ci bLT' -> 'MeK Alu Cip1MS0 Ci bLT'
[LEET] frase n.1 di READER-1: 'V3 wHKL QoBk y0u' -> 'Ve wHKL QoBk you'
[UPPER-FWD] frase n.1 di READER-2: 'MeK Alu Cip1MS0 Ci bLT' -> 'NeL Blu DipiNto Di blu'
[UPPER-FWD] frase n.1 di READER-1: 'Ve wHKL QoBk you' -> 'We wILL RoCk you'
[LEET] frase n.1 di READER-3: 'cGi U@ Oi@MN va l0n7@M0' -> 'cGi Ua OiaMN va lontaMo'
[UPPER-FWD] frase n.1 di READER-3: 'cGi Ua OiaMN va lontaMo' -> 'cHi Va PiaNO va lontaNo'
[TITLE-CASE] frase n.1 di READER-2: 'NeL Blu DipiNto Di blu' -> 'NeL Blu Dipinto Di Blu'
[READER-2] frase n.1 decodificata in: 'Nel Blu Dipinto Di Blu'
[READER-2] frase n.2: 'rTboe0 rvZMep PvZoCN'
...
[READER-1] frase n.1 decodificata in: 'We Will Rock You'
[READER-3] frase n.1 decodificata in: 'Chi Va Piano Va Lontano'
[READER-1] frase n.2 decodificata in: 'Here Comes The Sun'
[READER-3] frase n.2 decodificata in: 'Chi Dorme Non Piglia Pesci'
...
[READER-3] terminazione con 20 frasi decodificate
...
[LOWER-BACK] frase n.20 di READER-2: 'tuQbe@ g@B3MC0' -> 'stQad@ f@B3MC0'
[LEET] frase n.20 di READER-1: 'sNund Nf RHKDMcD' -> 'sNund Nf RHKDMcD'
[LEET] frase n.20 di READER-2: 'stQad@ f@B3MC0' -> 'stQada faBeMCo'
[UPPER-FWD] frase n.20 di READER-1: 'sNund Nf RHKDMcD' -> 'sOund Of SILENcE'
[UPPER-FWD] frase n.20 di READER-2: 'stQada faBeMCo' -> 'stRada faCeNDo'
[TITLE-CASE] frase n.20 di READER-1: 'sOund Of SILENcE' -> 'Sound Of Silence'
[READER-1] frase n.20 decodificata in: 'Sound Of Silence'
[TITLE-CASE] frase n.20 di READER-2: 'stRada faCeNDo' -> 'Strada Facendo'
[READER-1] terminazione con 20 frasi decodificate
[READER-2] frase n.20 decodificata in: 'Strada Facendo'
[READER-2] terminazione con 20 frasi decodificate
[LOWER-BACK] terminazione con 60 richieste servite
[LEET] terminazione con 60 richieste servite
[UPPER-FWD] terminazione con 60 richieste servite
[TITLE-CASE] terminazione con 60 richieste servite
[MAIN] riepilogo: encoded-list-1.txt: 20 frasi, encoded-list-2.txt: 20 frasi, encoded-list-3.txt: 20 frasi
[MAIN] terminazione
```